# Group 16: Database Design and ER Model

**Entities:**

Store
- store_id (PK)
- store_name
- street_address
- city
- state (FK to StateTax)
- zip_code
- Aisle_count

StateTax
- state (PK)
- tax_rate

Product
- sku (PK)
- product_name
- category
- tax_treatment (taxable/exempt)
- unit_price
- sold_by_weight (boolean)
- unit_of_measure (e.g. each, lb)

*Note: for weighted items, unit price is the price per pound

Inventory (composite primary key)
- store_id (PK, FK)
- sku (PK, FK)
- available_quantity
- aisle_number

Receipt
- receipt_id (PK)
- store_id (FK)
- receipt_date
- receipt_time
- subtotal
- tax_amount
- total_amount

ReceiptItem (composite primary key)
- receipt_id (PK, FK)
- receipt_line_number (PK)
- sku (FK)
- quantity
- weight (can be null)

- unit_price
- line_total

BankAccount
- account_id (PK)
- account_holder_name
- account_number (unique)
- account_balance

Payment
- payment_id (PK)
- receipt_id (FK, unique)
- account_id (FK, nullable for cash)
- payment_method (cash/card)
- amount
- payment_date
- payment_time
- payment_status (approved/declined)

**Explanation of tables and relationships**

The proposed database is designed to support the operations of a supermarket chain, with multiple stores located across multiple states. The database will manage store information, product details, inventory, state-specific pricing, customer purchases and payment processing. This model will also support each supermarket's checkout system where purchases are recorded and card payments are processed.

Tables and Their Purposes

1. StateTax
   This table stores the sales tax rate applicable to each state. Each state is uniquely identified by its state code, and its corresponding tax rate is used to calculate sales tax on taxable purchases at stores within that state.

2. Store
   This table stores information about each supermarket location. A store can be uniquely identified by its store_id, and possesses additional attributes including address and state.

3. Product
   Stores information about the products sold by the supermarket chain. Each product is identified by its SKU, and possess attributes including name, category, and tax treatment.

4. Inventory
   This entity tracks the products stocked at each store, including the available quantity and the aisle number. It resolves the many-to-many relationship between Store and Product, since a store carries multiple products and a product can be stocked at multiple stores.

5. Receipt
   This entity records all information about an individual purchase, including its receipt ID, the store where it occurred, the date, time, and the total amount. Each receipt represents one sale.

6. ReceiptItem
   Here, the individual product lines on a receipt are recorded. It includes the product's SKU, quantity or weight, unit price, and the line total. It connects and resolves the many-to-many relationship between Receipt and Product, allowing each receipt to contain multiple products and each product to appear on multiple receipts.

7. BankAccount
   This entity stores simulated bank account information, including a unique account_id, account holder's name, account number, and the current balance. It supports the processing of card payment transactions along the supermarket chain.

8. Payment
   This entity records payment information for a receipt. It includes the payment_id, receipt_id, payment method, amount, payment date and time, and payment status. A bank account is referenced for card payments only, since cash payments are not made from an account.

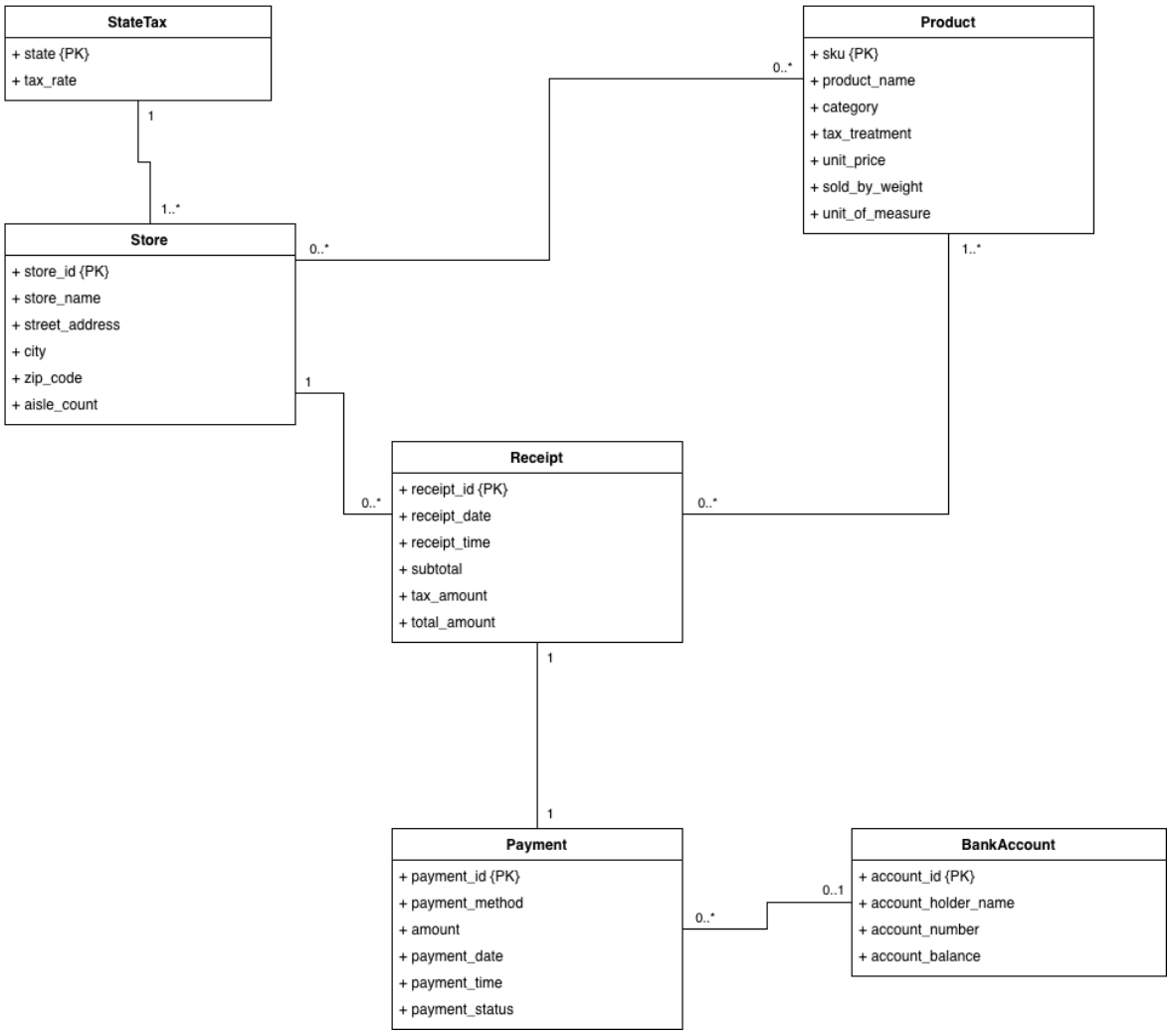
<u>Relationships and Cardinality</u>

| | |
|---|---|
| StateTax -> Store | 1:M |
| Store -> Inventory | 1:M |
| Product -> Inventory | 1:M |
| Store -> Receipt | 1:M |
| Receipt -> ReceiptItem | 1:M |
| Product -> ReceiptItem | 1:M |
| Receipt -> Payment | 1:1 |
| BankAccount -> Payment | 1:M |

## Entity Relationship Diagram (UML notation)

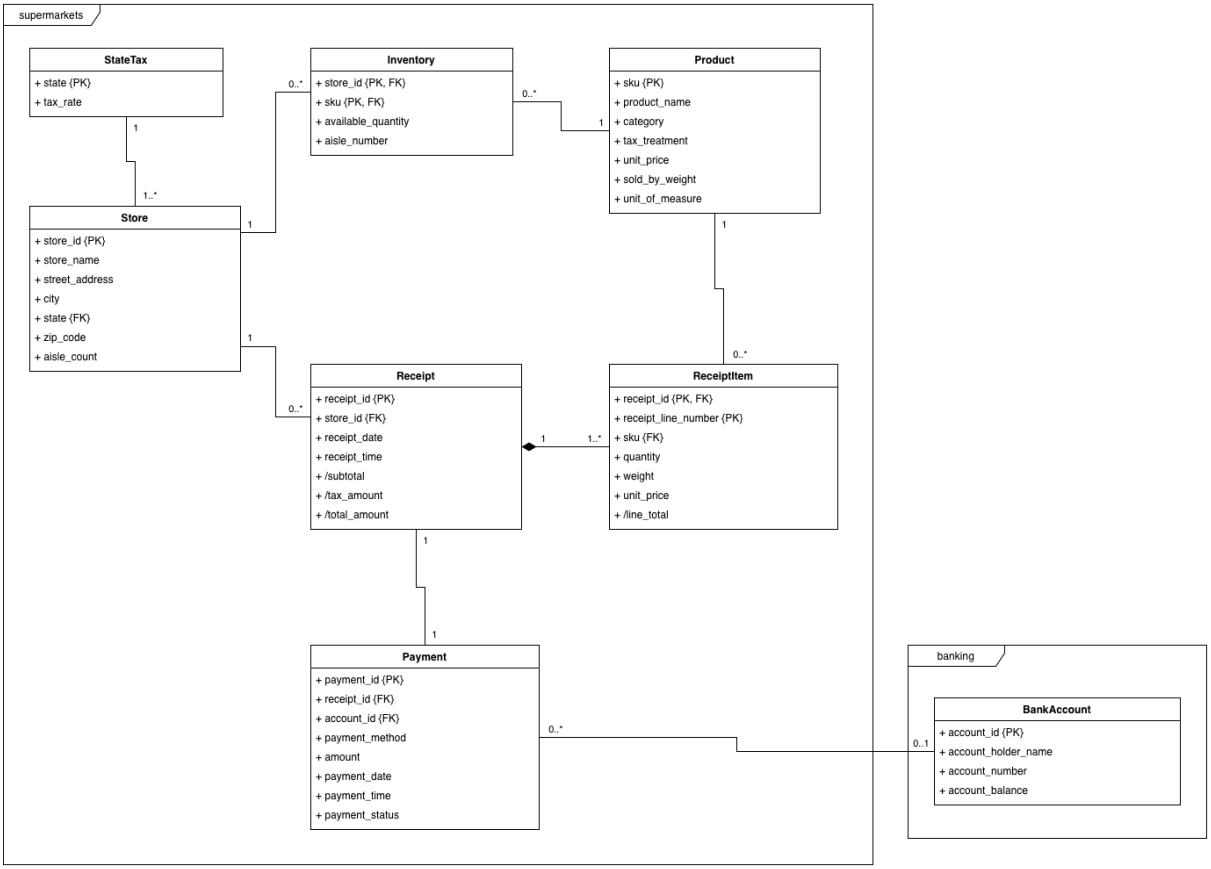
## Draft

**Group 16 - ER Diagram (Draft)**



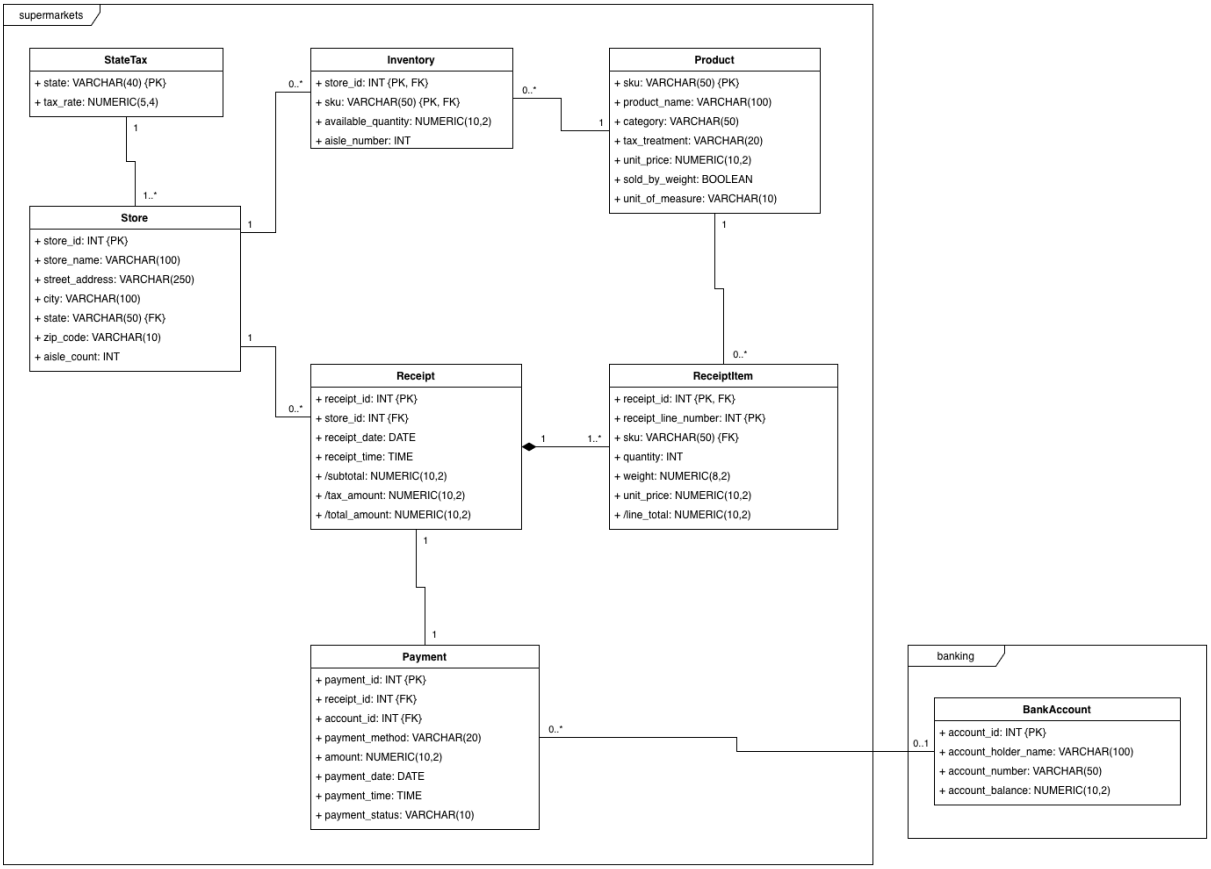
# Intermediate

**Group 16 - ER Diagram**



## Final

**Group 16 - Supermarket Chain ER Diagram**




### Transaction Processing

The supermarket management system implements checkout transactions using PostgreSQL to ensure that purchases are processed accurately and consistently. During checkout, the system retrieves product information, verifies inventory availability, and calculates the subtotal, applicable state sales tax, and total purchase amount. For card payments, the system verifies sufficient funds and deducts the purchase amount from the customer's bank account. Cash payments do not affect bank balances.

Each successful transaction creates a record in the Receipt table, stores individual purchased products in ReceiptItem, records payment details in Payment, and deducts purchased quantities or weights from Inventory. These operations are executed within a single PostgreSQL transaction using BEGIN and COMMIT. Inventory records are locked using SELECT ... FOR UPDATE to prevent conflicting updates during concurrent purchases. If a transaction fails due to insufficient funds, unavailable inventory, or another error, ROLLBACK prevents partial database changes and maintains data integrity.

**SQL Queries and JOIN Operations**

The system implements seven SQL queries to support supermarket reporting, inventory monitoring, payment analysis, and receipt retrieval.

Q1. Sales per Store
Uses a LEFT JOIN between Store and Receipt to calculate the number of receipts and total sales for each store within a specified date range. Stores without sales are included with a total of zero.

Q2. Top Products by Revenue
Joins ReceiptItem and Product to rank products by total revenue, displaying units sold, pounds sold, and product details.

Q3. Tax Collected per State
Joins StateTax, Store, and Receipt to calculate total sales subtotals and tax collected in each state.

Q4. Sales per State
Uses a LEFT JOIN between Store and Receipt to calculate the number of transactions and total revenue generated in each state.

Q5. Low-Stock Items per State
Joins Inventory, Store, and Product to identify products with fewer than 10 units or pounds remaining at each store.

Q6. Payments by Method
Uses GROUP BY to summarize the number and total value of payments according to payment method and status.

Q7. Full Receipt Lookup
Joins Receipt, Store, ReceiptItem, Product, and Payment to retrieve the details of a specific purchase, including purchased products, quantities, prices, totals, and payment method.

The complete SQL statements are provided in sql/query.sql.